

7. SubConsultas

Una subconsulta es una instrucción **SELECT** anidada dentro de una instrucción **SELECT**, **SELECT...INTO**, **INSERT...INTO**, **DELETE**, o **UPDATE** o dentro de otra subconsulta.

Puede utilizar tres formas de sintaxis para crear una subconsulta:

```
comparación [ANY | ALL | SOME]
(instrucción sql)
expresión [NOT] IN (instrucción sql)
[NOT] EXISTS (instrucción sql)
```

En donde:

comparación: Es una expresión y un operador de comparación que compara la expresión con el resultado de la subconsulta.

expresión: Es una expresión por la que se busca el conjunto resultante de la subconsulta.

instrucción

sql : Es una instrucción **SELECT**, que sigue el mismo formato y reglas que cualquier otra instrucción **SELECT**. Debe ir entre paréntesis.

Se puede utilizar una subconsulta en lugar de una expresión en la lista de campos de una instrucción **SELECT** o en una cláusula **WHERE** o **HAVING**.

En una subconsulta, se utiliza una instrucción **SELECT** para proporcionar un conjunto de uno o más valores especificados para evaluar en la expresión de la cláusula **WHERE** o **HAVING**.

Se puede utilizar el predicado **ANY** o **SOME**, los cuales son sinónimos, para recuperar registros de la consulta principal, que satisfagan la comparación con cualquier otro registro recuperado en la subconsulta. El ejemplo siguiente devuelve todos los productos cuyo precio unitario es mayor que el de cualquier producto vendido con un descuento igual o mayor al 25 por ciento.:

```
SELECT * FROM Productos WHERE PrecioUnidad > ANY
(SELECT PrecioUnidad FROM DetallePedido WHERE Descuento >= 0 .25);
```

El predicado **ALL** se utiliza para recuperar únicamente aquellos registros de la consulta principal que satisfacen la comparación con todos los registros recuperados en la subconsulta.

Si se cambia **ANY** por **ALL** en el ejemplo anterior, la consulta devolverá únicamente aquellos productos cuyo precio unitario sea mayor que el de todos los productos vendidos con un descuento igual o mayor al 25 por ciento. Esto es mucho **más restrictivo**.

El predicado **IN** se emplea para recuperar únicamente aquellos registros de la consulta principal para los que algunos registros de la subconsulta contienen un valor igual. El ejemplo siguiente devuelve todos los productos vendidos con un descuento igual o mayor al 25 por ciento.:

```
SELECT * FROM Productos WHERE IDProducto IN
(SELECT IDProducto FROM DetallePedido WHERE Descuento >= 0.25);
```

Inversamente se puede utilizar **NOT IN** para recuperar únicamente aquellos registros de la consulta principal para los que no hay ningún registro de la subconsulta que contenga un valor igual.

El predicado **EXISTS** (con la palabra reservada **NOT** opcional) se utiliza en comparaciones de verdad/falso para determinar si la subconsulta devuelve algún registro. Se puede utilizar también alias del nombre de la tabla en una subconsulta para referirse a tablas listadas en la cláusula **FROM** fuera de la subconsulta. El ejemplo siguiente devuelve los nombres de los empleados cuyo salario es igual o mayor que el salario medio de todos los empleados con el mismo título.

A la tabla **Empleados** se le ha dado el alias **T1**:

```
SELECT Apellido, Nombre, Titulo, Salario
FROM Empleados AS T1
WHERE Salario >= (SELECT Avg(Salario) FROM Empleados
WHERE T1.Titulo = Empleados.Titulo) ORDER BY Titulo;
```

Obtenga una lista con el nombre, cargo y salario de todos los agentes de ventas cuyo salario es mayor que el de todos los jefes y directores.

```
SELECT Apellidos, Nombre, Cargo, Salario
FROM Empleados
WHERE Cargo LIKE "Agente Ven*" AND Salario > ALL (SELECT Salario
FROM
Empleados WHERE (Cargo LIKE "*Jefe*") OR (Cargo LIKE "*Director*"));
```

Obtenga una lista con el nombre y el precio unitario de todos los productos con el mismo precio que el almíbar anisado.

```
SELECT DISTINCTROW NombreProducto, Precio_Unidad
FROM Productos
WHERE (Precio_Unidad = (SELECT Precio_Unidad FROM Productos WHERE
Nombre_Producto = "Almíbar anisado"));
```

Obtenga una lista de las compañías y los contactos de todos los clientes que han realizado un pedido en el segundo trimestre de 1993.

```
SELECT DISTINCTROW Nombre_Contacto,
Nombre_Compañia, Cargo_Contacto,
Telefono FROM Clientes WHERE (ID_Cliente IN (SELECT DISTINCTROW
ID_Cliente FROM Pedidos WHERE Fecha_Pedido >= #04/1/93# <#07/1/93#));
SELECT Nombre, Apellidos FROM Empleados
```

```
AS E WHERE EXISTS  
(SELECT * FROM Pedidos AS O WHERE O.ID_Empleado = E.ID_Empleado);
```

Selecciona el nombre de todos los empleados que han reservado al menos un pedido.

```
SELECT DISTINCTROW Pedidos.Id_Producto,  
Pedidos.Cantidad,  
(SELECT DISTINCTROW Productos.Nombre FROM Productos WHERE  
Productos.Id_Producto = Pedidos.Id_Producto) AS ElProducto FROM  
Pedidos WHERE Pedidos.Cantidad > 150 ORDER BY Pedidos.Id_Prod
```

Diferencias y similitudes de expresar una consulta mediante subconsultas anidadas o hacerlo mediante el operador JOIN

Un JOIN es una reunión entre dos tablas, es decir se realiza un producto cartesiano entre ellas y luego, si hay una condición, se eliminan las tuplas que no la cumplan. Una subconsulta, o subselect, es un subconjunto que, normalmente a partir de una condición o una función de agregación (como SUM, AVG, COUNT) participa de otra selección mayor. Se podría considerar un subselect como una consulta que sirve para obtener o completar otra consulta.

Evidentemente, en terminos de performance, crear la consulta utilizando JOINS suele ser mejor, debido a la cantidad de consultas involucradas. Por ejemplo:

```
SELECT A.col1  
  
FROM A JOIN B ON A.id = B.id
```

Representa una sola "pasada" por el montón de registros de nuestra base de datos. En cambio:

```
SELECT A.col1  
FROM A  
WHERE A.id IN (SELECT B.id FROM B)
```

Estaríamos hablando de $N + 1$ consultas a realizar. Creo que se logra ver la diferencia, igualmente con el uso de índices y dependiendo del planificador (o del tipo de base de datos) ésta diferencia no es tan grande.

Por otro lado a veces los subselect son la forma más cómoda de escribir una consulta compleja, y otras veces son la única manera de lograrlas.

Subconsultas con in

<http://www.postgresqlya.com.ar/temarios/descripcion.php?cod=218&punto=60>

Vimos que una subconsulta puede reemplazar una expresión. Dicha subconsulta debe devolver un valor escalar o una lista de valores de un campo; las subconsultas que retornan una lista de valores reemplazan a una expresión en una cláusula "where" que contiene la palabra clave "in".

El resultado de una subconsulta con "in" (o "not in") es una lista. Luego que la subconsulta retorna resultados, la consulta exterior los usa.

La sintaxis básica es la siguiente:

```
...where EXPRESION in (SUBCONSULTA);
```

Este ejemplo muestra los nombres de las editoriales que han publicado libros de un determinado autor:

```
select nombre
  from editoriales
 where codigo in
    (select codigoeditorial
      from libros
      where autor='Richard Bach');
```

La subconsulta (consulta interna) retorna una lista de valores de un solo campo (codigo) que la consulta exterior luego emplea al recuperar los datos.

Podemos reemplazar por un "join" la consulta anterior:

```
select distinct nombre
  from editoriales as e
 join libros
 on codigoeditorial=e.codigo
 where autor='Richard Bach';
```

Una combinación (join) siempre puede ser expresada como una subconsulta; pero una subconsulta no siempre puede reemplazarse por una combinación que retorne el mismo resultado. Si es posible, es aconsejable emplear combinaciones en lugar de subconsultas, son más eficientes.

Se recomienda probar las subconsultas antes de incluirlas en una consulta exterior, así puede verificar que retorna lo necesario, porque a veces resulta difícil verlo en consultas anidadas.

También podemos buscar valores No coincidentes con una lista de valores que retorna una subconsulta; por ejemplo, las editoriales que no han publicado libros de un autor específico:

```
select nombre
  from editoriales
 where codigo not in
    (select codigoeditorial
     from libros
     where autor='Richard Bach');
```

Subconsultas (Exists y No Exists)

<http://www.postgresqlya.com.ar/temarios/descripcion.php?cod=221&punto=63>

Los operadores "exists" y "not exists" se emplean para determinar si hay o no datos en una lista de valores.

Estos operadores pueden emplearse con subconsultas correlacionadas para restringir el resultado de una consulta exterior a los registros que cumplen la subconsulta (consulta interior). Estos operadores retornan "true" (si las subconsultas retornan registros) o "false" (si las subconsultas no retornan registros).

Cuando se coloca en una subconsulta el operador "exists", PostgreSQL analiza si hay datos que coinciden con la subconsulta, no se devuelve ningún registro, es como un test de existencia; PostgreSQL termina la recuperación de registros cuando por lo menos un registro cumple la condición "where" de la subconsulta.

La sintaxis básica es la siguiente:

```
... where exists (SUBCONSULTA);
```

En este ejemplo se usa una subconsulta correlacionada con un operador "exists" en la cláusula "where" para devolver una lista de clientes que compraron el artículo "lapiz":

```
select cliente,numero
  from facturas as f
 where exists
    (select *from Detalles as d
     where f.numero=d.numerofactura
     and d.articulo='lapiz');
```

Puede obtener el mismo resultado empleando una combinación.

Podemos buscar los clientes que no han adquirido el artículo "lapiz" empleando "if not exists":

```
select cliente,numero
from facturas as f
where not exists
(select *from Detalles as d
where f.numero=d.numerofactura
and d.articulo='lapiz');
```

61 - Subconsultas any - some - all

<http://www.postgresqlya.com.ar/temarios/descripcion.php?cod=219&punto=61>

"any" y "some" son sinónimos. Chequean si alguna fila de la lista resultado de una subconsulta se encuentra el valor especificado en la condición.

Compara un valor escalar con los valores de un campo y devuelven "true" si la comparación con cada valor de la lista de la subconsulta es verdadera, sino "false".

El tipo de datos que se comparan deben ser compatibles.

La sintaxis básica es:

```
...VALORESCALAR OPERADORDECOMPARACION
ANY (SUBCONSULTA);
```

Queremos saber los títulos de los libros de "Borges" que pertenecen a editoriales que han publicado también libros de "Richard Bach", es decir, si los libros de "Borges" coinciden con ALGUNA de las editoriales que publicó libros de "Richard Bach":

```
select titulo
from libros
where autor='Borges' and
codigoeditorial = any
(select e.codigo
from editoriales as e
join libros as l
on codigoeditorial=e.codigo
where l.autor='Richard Bach');
```

La consulta interna (subconsulta) retorna una lista de valores de un solo campo (puede ejecutar la subconsulta como una consulta para probarla), luego, la consulta externa compara cada valor de "codigoeditorial" con cada valor de la lista devolviendo los títulos de "Borges" que coinciden.

"all" también compara un valor escalar con una serie de valores. Chequea si TODOS los valores de la lista de la consulta externa se encuentran en la lista de valores devuelta por la

consulta
Sintaxis:

interna.

```
VALORESCALAR OPERADORDECOMPARACION all (SUBCONSULTA);
```

Queremos saber si TODAS las editoriales que publicaron libros de "Borges" coinciden con TODAS las editoriales que publicaron libros de "Richard Bach":

```
select titulo
from libros
where autor='Borges' and
codigoeditorial = all
(select e.codigo
 from editoriales as e
 join libros as l
 on codigoeditorial=e.codigo
 where l.autor='Richard Bach');
```

La consulta interna (subconsulta) retorna una lista de valores de un solo campo (puede ejecutar la subconsulta como una consulta para probarla), luego, la consulta externa compara cada valor de "codigoeditorial" con cada valor de la lista, si TODOS coinciden, devuelve los títulos.

Veamos otro ejemplo con un operador de comparación diferente:

Queremos saber si ALGUN precio de los libros de "Borges" es mayor a ALGUN precio de los libros de "Richard Bach":

```
select titulo,precio
from libros
where autor='Borges' and
precio > any
(select precio
 from libros
 where autor='Bach');
```

El precio de cada libro de "Borges" es comparado con cada valor de la lista de valores retornada por la subconsulta; si ALGUNO cumple la condición, es decir, es mayor a ALGUN precio de "Richard Bach", se lista.

Veamos la diferencia si empleamos "all" en lugar de "any":

```
select titulo,precio
from libros
where autor='borges' and
precio > all
(select precio
 from libros
 where autor='bach');
```

El precio de cada libro de "Borges" es comparado con cada valor de la lista de valores retornada por la subconsulta; si cumple la condición, es decir, si es mayor a TODOS los precios de "Richard Bach" (o al mayor), se lista.

Emplear "= any" es lo mismo que emplear "in".

Emplear "<> all" es lo mismo que emplear "not in".

Recuerde que solamente las subconsultas luego de un operador de comparación al cual es seguido por "any" o "all") pueden incluir cláusulas "group by".